UNIVERSIDAD NACIONAL DE INGENIERÍA — RECINTO UNIVERSITARIO SIMÓN BOLÍVAR

Área de Conocimiento: Tecnología de la Información y la Comunicación

Programa Académico de Ingeniería en Sistemas

Asignatura: Programación de Aplicaciones de Escritorio

Documento Técnico y Manual de Usuario del Proyecto Sistema de Gestión Académica

Nombre completo	Carné	Rol en el proyecto
Samantha Carolina Alonso Rodriguez		
Josnny Saray Hernandez Cardenas		
Xochilt Vanessa Cruz Soza		

Docente (revisor): Walter José Quintero Acevedo

Periodo académico: II Semestre 2026

Versión del documento: v1.0 — corresponde al software GestionAcademica v1.0

Fecha de entrega: viernes 13 de noviembre de 2026

Declaratoria de autoría

Los firmantes declaran bajo juramento que este documento es de su autoría, que su contenido corresponde al sistema ejecutable entregado y que las fuentes consultadas han sido citadas.

Samantha C. Alonso R.	Josnny S. Hernandez C.	Xochilt V. Cruz S.

Índice de Contenidos

Parte I. Documento Técnico 1
1. Resumen ejecutivo 1
2. Descripción del problema y justificación 1
3. Objetivos 2
4. Alcance y limitaciones 2
5. Análisis del proyecto 3
6. Arquitectura del sistema 4
7. Modelado UML 5
8. Modelo de datos 9
9. Programación concurrente 10
10. Consumo de servicios web REST 11
11. Generación de reportes 12
12. Manejo de errores y Logger 13
13. Pruebas realizadas 13
14. Conclusiones técnicas 14

Parte II. Manual de Usuario 16
15. Introducción y público objetivo 16
16. Requisitos del sistema 16
17. Procedimiento de instalación 16
18. Credenciales de acceso y roles 17
19. Guía de navegación por pantallas 17
20. Casos de uso paso a paso 20
21. Errores comunes y soporte 22
22. Glosario de términos 23
23. Preguntas frecuentes 23

Bibliografía 24

Control de cambios

Versión	Fecha	Sección afectada	Descripción del cambio
v0.5		Parte I	Avances E1 a E7 del documento técnico
v1.0		Todo el documento	Unión del Documento Técnico y el Manual de Usuario (Entregable 8)

Parte I. Documento Técnico

1. Resumen ejecutivo

El Sistema de Gestión Académica es una aplicación de escritorio para Windows, desarrollada en C# sobre .NET 8 y Windows Forms, que permite a un centro educativo administrar estudiantes, sesiones de asistencia y evaluaciones finales desde una sola herramienta, en sustitución de registros dispersos y cálculos manuales de la nota final. La información se guarda en SQL Server Express, en una base de datos denominada «Gestion Academica», con tablas relacionadas por llaves foráneas y 21 procedimientos almacenados. La aplicación se organiza en cuatro capas: presentación, negocio, acceso a datos y entidades. El acceso a datos combina ADO.NET, con consultas parametrizadas y transacciones, y Entity Framework Core con consultas LINQ para los reportes. Las operaciones largas se ejecutan en segundo plano con Task, Thread, CancellationToken e IProgress, de modo que la interfaz permanece responsiva. El sistema permite crear cuentas de usuario con contraseñas protegidas mediante PBKDF2 y consume dos servicios REST mediante HttpClient: Nager.Date, para consultar feriados y cruzarlos con las sesiones registradas, y Resend, para enviar notificaciones por correo. Como resultado se obtuvo un sistema funcional con inicio de sesión y registro de cuentas, tres módulos de mantenimiento con validaciones, un módulo de procesos concurrentes, reportes dinámicos exportables a CSV y un módulo de servicios web. Entre las limitaciones reconocidas se encuentran la ausencia de roles, de inyección de dependencias, de bitácora de errores en archivo y de reportes en PDF.

2. Descripción del problema y justificación

En un centro educativo se relacionan de forma permanente tres tipos de información: los estudiantes matriculados, las sesiones de clase a las que asisten y las evaluaciones finales que resumen su rendimiento por materia. Cuando esa información se lleva en cuadernos u hojas de cálculo separadas aparecen datos duplicados o inconsistentes, dificultad para consultar el historial de un estudiante y errores en el cálculo de la nota final.

El sistema centraliza los datos en una base de datos relacional con integridad referencial y ofrece formularios con validaciones que impiden registros incompletos o fuera de rango (por ejemplo, sesiones con fecha futura o puntajes superiores a 100). La nota final se calcula automáticamente como la suma de los puntos de asistencia, participación y pruebas.

El actor principal es el **usuario autenticado** (personal administrativo o docente). Los actores externos son los servicios **Nager.Date** y **Resend**. Se eligió una aplicación de escritorio porque el entorno previsto es una oficina con equipo Windows y SQL Server local, Windows Forms ofrece un modelo de eventos con diseñador visual apropiado para formularios con rejillas de datos (Microsoft, s. f.-b) y la base de datos no se expone a Internet. Como contrapartida, debe instalarse en cada equipo y no es accesible desde dispositivos móviles.

3. Objetivos

Objetivo general. Desarrollar una aplicación de escritorio para la gestión académica que persista sus datos en SQL Server, siga una arquitectura en capas y aplique programación concurrente, acceso a datos con ORM e integración con servicios web REST.

Objetivos específicos:

1. Diseñar una interfaz con Windows Forms que incluya inicio de sesión, registro de cuentas con contraseñas protegidas, menú principal y formularios con validaciones.
2. Implementar el CRUD de estudiantes, sesiones y evaluaciones finales con ADO.NET, consultas parametrizadas, procedimientos almacenados y transacciones.
3. Separar la aplicación en capas de presentación, negocio, acceso a datos y entidades, sin SQL ni reglas de negocio en los formularios.
4. Ejecutar procesos largos en segundo plano con Task y Thread, con progreso y cancelación cooperativa.
5. Complementar el acceso a datos con Entity Framework Core y LINQ, y generar reportes dinámicos exportables a CSV.
6. Consumir servicios REST con HttpClient y System.Text.Json (feriados y notificaciones por correo) e integrar la información con la base de datos.

4. Alcance y limitaciones

El sistema autentica usuarios y permite crear cuentas nuevas; registra, modifica, elimina y busca estudiantes, sesiones y evaluaciones; calcula la nota final; ejecuta cargas concurrentes con cancelación; gestiona estudiantes con EF Core y genera tres reportes con LINQ; consulta feriados

y los cruza con las sesiones; y envía notificaciones manuales por correo. La Tabla 1 declara lo que el sistema **no** hace, conforme a la exigencia de honestidad de la guía.

Carencia	Situación en la versión entregada
Roles y permisos	Todas las cuentas tienen los mismos permisos; no hay perfiles diferenciados.
Inyección de dependencias / DIP	No se usa Microsoft.Extensions.DependencyInjection; las clases se crean con new y la capa de negocio depende de clases concretas (DAO).
Bitácora de errores (Logger)	No existe registro de errores en archivo; los errores se muestran con MessageBox.
Reportes en PDF	Los reportes se muestran en pantalla y se exportan a CSV; no se genera PDF.
Migraciones EF Core	EF Core se usa sobre la base creada con scripts T-SQL (Fluent API); no se emplea Code-First.
Cadena de conexión	Fija en DataAccess/Conexion.cs (localhost\SQLEXPRESS, autenticación de Windows); no se lee de appsettings.json.
Biblioteca de UI moderna	No se usa MaterialSkin; el aspecto se define en Estilo/TemaVisual.cs con controles estándar.
Pruebas automatizadas	No hay pruebas unitarias; las pruebas son manuales (capítulo 13).
Contraseña del usuario inicial	Las cuentas nuevas usan PBKDF2 con sal; el usuario inicial «UNI» conserva SHA-256 sin sal por compatibilidad. No hay pantalla para cambiar contraseña.
Notificaciones por correo	Son manuales: el esquema de estudiantes no guarda correo electrónico, por lo que no se envían automáticamente.
Consistencia de tipos EF Core	El script define nota_final como FLOAT y estado como INT, mientras el DbContext aplica conversiones (decimal y texto); debe verificarse contra la base real.

Tabla 1. Carencias y limitaciones reconocidas
Fuente: elaboración propia.

Limitaciones técnicas y supuestos. La aplicación solo funciona en Windows 10 o superior con SQL Server; el consumo de las APIs requiere Internet y depende de terceros; se asume una instancia local SQLEXPRESS con la base creada con el script del proyecto; y la pausa simulada de 2500 ms del módulo concurrente (PausaSimuladaMs) es solo demostrativa y se desactiva con el valor 0.

5. Análisis del proyecto

La Tabla 2 lista los requerimientos funcionales y la Tabla 3 los no funcionales; son el insumo de los casos de prueba del capítulo 13.

Cód.	Descripción	Actor	Prior.	Criterio de aceptación
RF-01	Autenticar usuarios contra la base de datos.	Usuario	Alta	Credenciales válidas abren el menú; inválidas muestran «Usuario o contraseña incorrectos.»
RF-02	Crear cuentas nuevas con validación de datos.	Usuario	Alta	Se crea la cuenta y puede iniciar sesión; datos inválidos o usuario repetido se rechazan.
RF-03	Registrar, modificar, eliminar y buscar estudiantes.	Usuario autenticado	Alta	Los cambios se reflejan en la rejilla; eliminar exige confirmación.

Cód.	Descripción	Actor	Prior.	Criterio de aceptación
RF-04	Registrar, modificar, eliminar y buscar sesiones de asistencia.	Usuario autenticado	Alta	No se acepta fecha futura.
RF-05	Registrar, modificar, eliminar y buscar evaluaciones con nota automática.	Usuario autenticado	Alta	Nota final = puntos de asistencia + participación + pruebas; valores de 0 a 100.
RF-06	Ejecutar y cancelar la carga concurrente de datos.	Usuario autenticado	Media	La barra llega a 100 % sin congelar la ventana; al cancelar se informa.
RF-07	Gestionar estudiantes con Entity Framework Core.	Usuario autenticado	Media	Cada operación se confirma con un mensaje.
RF-08	Generar tres reportes dinámicos con LINQ y filtros.	Usuario autenticado	Alta	Se muestra «Registros encontrados» y los datos filtrados.
RF-09	Exportar reportes y feriados a CSV.	Usuario autenticado	Media	Se genera un CSV que abre en Excel.
RF-10	Consultar feriados por país y año.	Usuario autenticado	Media	Se muestra la tabla, el próximo feriado y el JSON recibido.
RF-11	Cruzar feriados con las sesiones registradas.	Usuario autenticado	Baja	Se listan las sesiones que coinciden con un feriado.
RF-12	Enviar notificaciones por correo (API Resend).	Usuario autenticado	Baja	El servicio acepta la solicitud y se muestra su identificador.
RF-13	Cerrar sesión.	Usuario autenticado	Baja	Se regresa al inicio de sesión.

Tabla 2. Requerimientos funcionales
Fuente: elaboración propia.

Cód.	Categoría	Descripción	Prior.	Criterio de aceptación
RNF-01	Rendimiento	Las operaciones largas no bloquean la interfaz.	Alta	La ventana responde durante cargas, reportes y consultas web.
RNF-02	Rendimiento	Las llamadas HTTP tienen tiempo límite.	Media	15 s (feriados) y 30 s (correo); luego se informa el error.
RNF-03	Seguridad	Contraseñas nuevas protegidas con PBKDF2 y sal.	Alta	El hash guardado tiene el formato PBKDF2$iteraciones$sal$hash.
RNF-04	Seguridad	Consultas parametrizadas y escrituras atómicas.	Alta	Sin concatenación de texto en SQL; SqlTransaction con commit/rollback.
RNF-05	Seguridad	La clave de la API de correo no está en el código.	Alta	Se lee de la variable de entorno RESEND_API_KEY.
RNF-06	Usabilidad	Mensajes claros y confirmación al eliminar.	Media	Toda validación fallida muestra un mensaje.
RNF-07	Mantenibilidad	Separación en capas y conexión centralizada.	Alta	Formularios sin SQL; cadena de conexión en un solo lugar.
RNF-08	Portabilidad	Windows 10 o superior con .NET 8.	Media	Inicia con .NET 8 Desktop Runtime y SQL Server.

Tabla 3. Requerimientos no funcionales
Fuente: elaboración propia.

6. Arquitectura del sistema

El sistema adopta una arquitectura en capas, que organiza el código en niveles con responsabilidades diferenciadas y dependencias en un solo sentido (Fowler, 2002; Martin, 2018). La Figura 1 muestra las capas, sus carpetas y la dirección de las dependencias: la presentación

invoca al negocio; el negocio invoca al acceso a datos y a los servicios externos; las entidades son transversales. Una regla del proyecto es que la interfaz nunca ejecuta SQL y la base de datos nunca conoce a la interfaz.

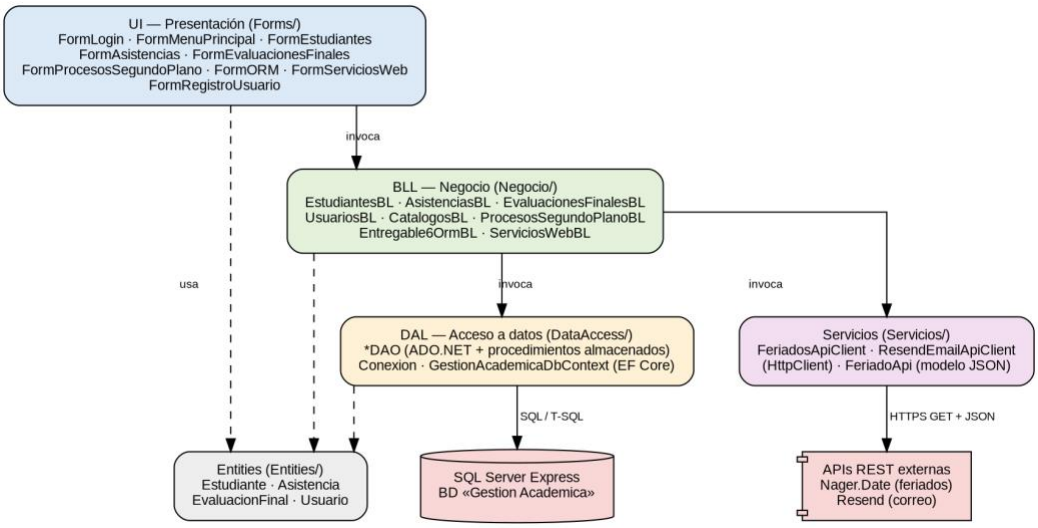


Figura 1. Arquitectura en capas del sistema
Fuente: elaboración propia a partir del código del proyecto.

Patrón. No se implementa MVP ni MVVM: la presentación usa el modelo de eventos de Windows Forms con code-behind y delega en la capa de negocio toda validación y regla. Dado el tamaño del dominio (tres entidades principales y cuatro catálogos), la arquitectura en capas es suficiente, y permitió añadir EF Core, el consumo REST y el registro de usuarios sin alterar los módulos existentes.

Inyección de dependencias y DIP. No se implementó inyección de dependencias: cada clase crea a sus colaboradores, por ejemplo private readonly EstudiantesDAO estudiantesDAO = new EstudiantesDAO(); en EstudiantesBL. Por ello el principio de inversión de dependencias no se cumple plenamente, pues el negocio depende de una clase concreta y no de una abstracción (Martin, 2018). Como mejora, bastaría definir interfaces para los DAO y registrarlas con Microsoft.Extensions.DependencyInjection en Program.cs.

Despliegue. La aplicación se instala en un equipo Windows con el .NET 8 Desktop Runtime y se conecta a SQL Server Express local (localhost\SQLEXPRESS, autenticación de Windows). De forma opcional consulta por HTTPS los servicios date.nager.at y api.resend.com.

7. Modelado UML

La Figura 2 presenta los casos de uso: el actor principal es el usuario autenticado y los actores externos son las APIs. Las relaciones «extend» indican comportamientos opcionales y «include» que la consulta de feriados siempre incorpora el cruce con las sesiones.

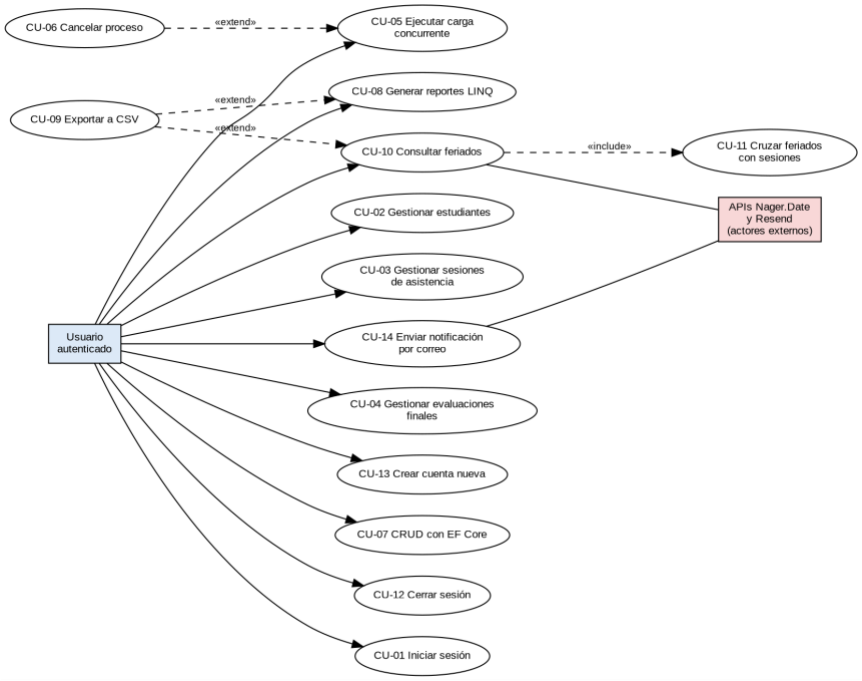


Figura 2. Diagrama de casos de uso
Fuente: elaboración propia a partir del código del proyecto.

La Figura 3 muestra las entidades y las clases principales de cada capa con atributos, métodos y modificadores de acceso (+ público, - privado, # protegido). El módulo de estudiantes se detalla como ejemplo del flujo formulario–negocio–DAO; sesiones y evaluaciones siguen la misma estructura.

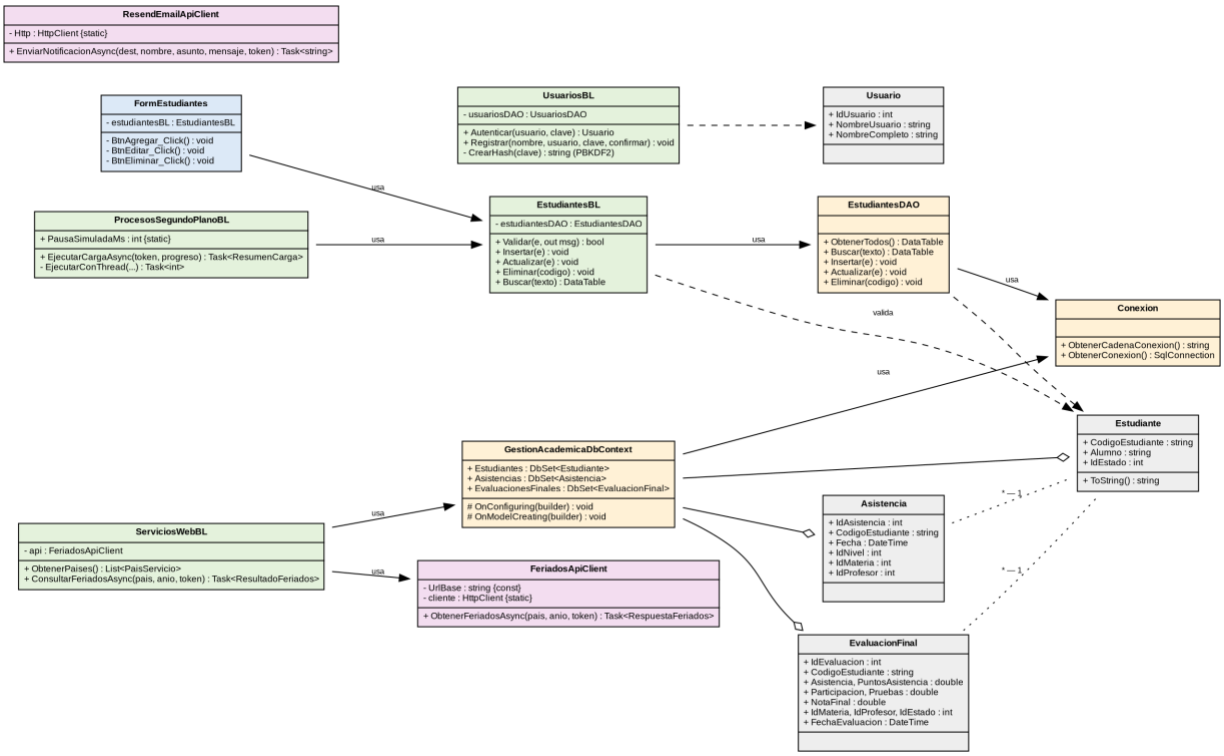


Figura 3. Diagrama de clases (entidades y clases principales)
Fuente: elaboración propia a partir del código del proyecto.

Las transacciones críticas se muestran en las figuras 4 y 5: el registro de un estudiante, que recorre las tres capas y termina en una transacción SQL, y la consulta de feriados, que combina un servicio externo con una consulta LINQ.

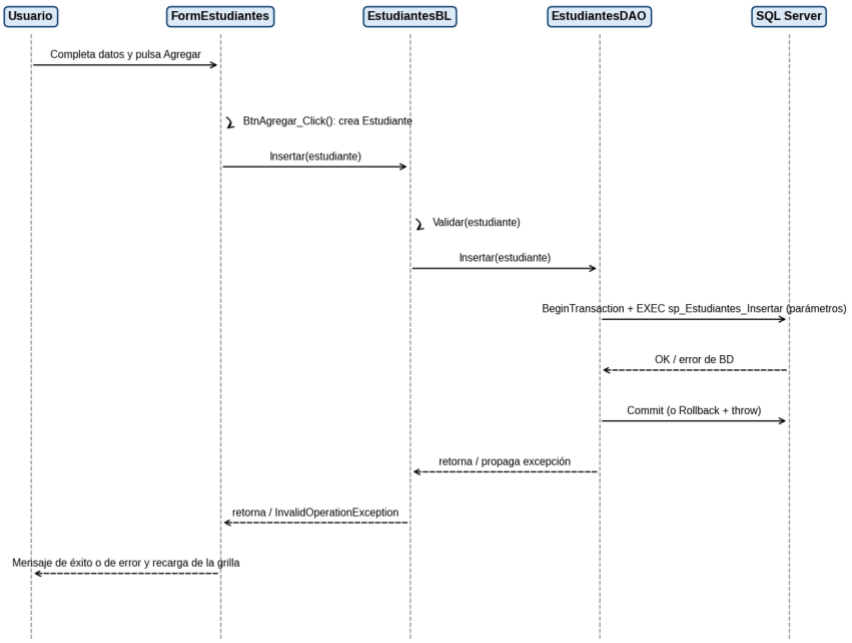


Figura 4. Diagrama de secuencia: registro de un estudiante
Fuente: elaboración propia a partir del código del proyecto.

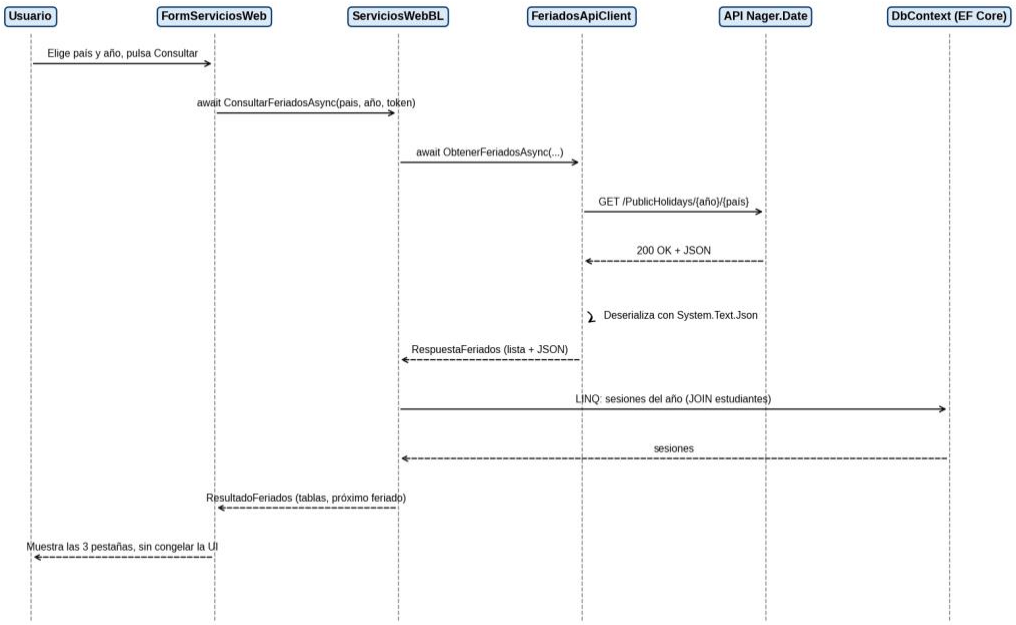


Figura 5. Diagrama de secuencia: consulta de feriados
Fuente: elaboración propia a partir del código del proyecto.

8. Modelo de datos

La Figura 6 muestra el modelo entidad-relación de la base «Gestion Academica». Las cuatro tablas de catálogo (Estados, Niveles, Materias, Profesores) se relacionan uno a muchos con las tablas transaccionales; Usuarios es independiente y sirve al inicio de sesión.

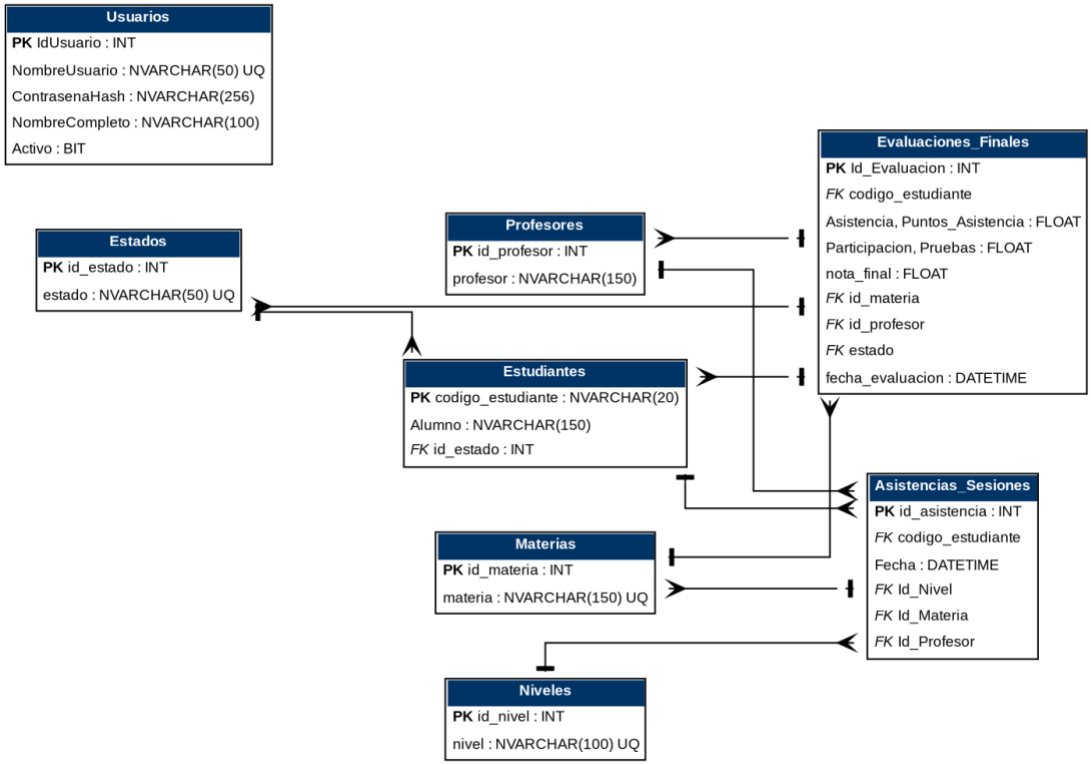


Figura 6. Diagrama entidad-relación de la base de datos
Fuente: elaboración propia a partir del código del proyecto.

8.1 Modelo lógico

El script **Database/Script_Instalacion_Completo.sql** crea las tablas que faltan, los datos mínimos de catálogo, el usuario inicial y los procedimientos almacenados, sin borrar datos (cada creación se protege con IF OBJECT_ID ... IS NULL). Se entrega como archivo del proyecto y es ejecutable sin modificaciones (Microsoft, s. f.-a). Extracto:

```
CREATE TABLE dbo.Estudiantes (
   codigo_estudiante NVARCHAR(20)  NOT NULL CONSTRAINT PK_Estudiantes PRIMARY KEY,
   Alumno            NVARCHAR(150) NOT NULL,
   id_estado         INT           NOT NULL,
   CONSTRAINT FK_Estudiantes_Estados FOREIGN KEY (id_estado)
      REFERENCES dbo.Estados(id_estado));
```

El acceso ADO.NET usa 21 procedimientos almacenados con parámetros (6 de estudiantes, 5 de asistencias, 5 de evaluaciones, 4 de catálogos y 1 de seguridad). La consulta de inicio de sesión y las eliminaciones adicionales de EstudiantesDAO usan consultas fijas con parámetros. El script

no define restricciones CHECK; los rangos se validan en la capa de negocio. La columna Usuarios.ContrasenaHash es NVARCHAR(256) para alojar hashes PBKDF2; las bases creadas con una versión anterior se actualizan con Database/Actualizar_Seguridad_Usuarios.sql.

8.2 Modelo físico: Entity Framework Core

Entity Framework Core 8.0.19 se usa como complemento de ADO.NET (Microsoft, 2024b): GestionAcademicaDbContext mapea las tablas existentes con Fluent API en OnModelCreating y reutiliza la cadena de conexión de la clase Conexion mediante OnConfiguring. No se emplean migraciones (Add-Migration, Update-Database): el esquema lo crea el script T-SQL y EF Core solo lo consume. La consistencia referencial la garantizan las llaves foráneas de SQL Server; por ejemplo, eliminar un estudiante con registros relacionados produce un error de base de datos que la aplicación captura y muestra con un mensaje claro.

```
modelBuilder.Entity<Estudiante>(entity =>
{
    entity.ToTable("Estudiantes");
    entity.HasKey(e => e.CodigoEstudiante);
    entity.Property(e => e.CodigoEstudiante).HasColumnName("codigo  estudiante").HasMaxLength(20);
    entity.Property(e => e.Alumno).HasMaxLength(150).IsRequired();
});
```

9. Programación concurrente

El módulo «Programación concurrente» (FormProcesosSegundoPlano y ProcesosSegundoPlanoBL) ejecuta operaciones largas sin bloquear el hilo de la interfaz, siguiendo los patrones asincrónicos de .NET (Microsoft, 2024c). Al pulsar «Iniciar proceso» se lanzan tres cargas simultáneas (Tabla 4) que se esperan con Task.WhenAll.

Carga	Mecanismo	Detalle
Estudiantes	Thread dedicado	Hilo propio (IsBackground = true) que notifica con un TaskCompletionSource.
Sesiones de asistencia	Task.Run	Tarea del ThreadPool con CancellationToken.
Evaluaciones finales	Task.Run	Tarea del ThreadPool con CancellationToken.

Tabla 4. Cargas concurrentes y mecanismo empleado
Fuente: elaboración propia.

```
Task<int> tareaAsistencias = Task.Run(() =>
{
    cancellationToken.ThrowIfCancellationRequested();
    DataTable tabla = asistenciasBL.ObtenerTodos();
    int total = Interlocked.Increment(ref completadas);
    progreso?.Report(total * 30);
    return tabla.Rows.Count;
}, cancellationToken);
int[] resultados = await Task.WhenAll(tareaEstudiantes, tareaAsistencias, tareaEvaluaciones);
```

async/await. BtnIniciar_Click es async void (válido para manejadores de eventos) y espera con await, por lo que el hilo de la interfaz queda libre; igual criterio se aplica a la consulta de feriados, al envío de correo y a los reportes (Task.Run). **Cancelación.** Un CancellationTokenSource permite detener el proceso con «Cancelar»; las tareas revisan el token y terminan con OperationCanceledException, y la pausa simulada es interrumpible. **Sincronización y progreso.** El contador de tareas terminadas usa Interlocked.Increment, por lo que no se requirieron lock ni SemaphoreSlim; el avance se informa con IProgress<int>, que actualiza la barra en el hilo de la interfaz. No se usa Application.DoEvents(). La evidencia visual está en las figuras 13 y 14 del manual.

10. Consumo de servicios web REST

El sistema consume dos servicios con HttpClient. **Nager.Date** (Nager.Date, s. f.), público y sin clave, devuelve los feriados de un país y año, que se cruzan con las sesiones registradas. **Resend** (Resend, s. f.) envía notificaciones por correo y requiere una clave API. La Tabla 5 resume ambos.

Elemento	Nager.Date	Resend
URL base	https://date.nager.at/api/v3/	https://api.resend.com/
Endpoint	GET PublicHolidays/{año}/{país}	POST emails
Autenticación	Ninguna	Bearer con la variable de entorno RESEND_API_KEY
Tiempo límite	15 s	30 s
Clase	FeriadosApiClient	ResendEmailApiClient

Tabla 5. Servicios REST consumidos
Fuente: elaboración propia.

Cada clase mantiene un único HttpClient estático y reutilizable, práctica que evita el agotamiento de sockets (Microsoft, 2024a); no se usa IHttpClientFactory. Fragmento de FeriadosApiClient:

```
string ruta = $"PublicHolidays/{anio}/{codigoPais.Trim().ToUpperInvariant()}";
using (HttpResponseMessage respuesta = await cliente.GetAsync(ruta, cancellationToken))
{
   respuesta.EnsureSuccessStatusCode();
   string json = await respuesta.Content.ReadAsStringAsync(cancellationToken);
   List<FeriadoApi> feriados = JsonSerializer.Deserialize<List<FeriadoApi>>(json, opcionesLectura)
                               ?? new List<FeriadoApi>();
}
```

La respuesta es un arreglo JSON que se deserializa con System.Text.Json hacia la clase FeriadoApi mediante [JsonPropertyName]; los campos no mapeados se ignoran. Ejemplo de un elemento:

```
{ "date": "2026-09-14", "localName": "Batalla de San Jacinto",
  "name": "Battle of San Jacinto", "countryCode": "NI", "global": true, "types": ["Public"] }
```

ResendEmailApiClient valida el correo y la longitud de los campos, codifica el texto en HTML, agrega la cabecera Authorization: Bearer con la clave leída de la variable de entorno y envía el cuerpo JSON (remitente, destinatario, asunto, texto y html). Si no existe RESEND_FROM_EMAIL usa onboarding@resend.dev. Un código 200 indica que el proveedor aceptó la solicitud, no que el mensaje llegó a la bandeja. La Tabla 6 resume el manejo de errores; no hay reintentos, y si falla el cruce con la base de datos los feriados se muestran igualmente con un aviso (degradación elegante).

Situación	Tratamiento
Sin conexión (HttpRequestException)	«No se pudo conectar con el servicio web. Verifique su conexión a Internet.»
Tiempo agotado (TaskCanceledException)	«El servicio web tardó demasiado en responder (tiempo agotado).»
HTTP 404 o 204 (sin datos)	Mensaje que sugiere probar otro país o año.
Otros 4xx / 5xx	EnsureSuccessStatusCode lanza la excepción; en Resend se muestra el código y el detalle del servicio.
JSON inválido (JsonException)	«La respuesta del servicio no tiene el formato JSON esperado.»
Falta RESEND_API_KEY	«No está configurada la variable de entorno RESEND_API_KEY…»

Tabla 6. Manejo de errores en el consumo de servicios web
Fuente: elaboración propia.

11. Generación de reportes

Los reportes se generan en FormORM (pestaña «Reportes dinámicos con LINQ») y se exportan a **CSV** con separador punto y coma, compatible con Excel; no se genera PDF ni se usó una biblioteca de reportes. La Tabla 7 describe los tres reportes, todos con LINQ sobre el DbContext, filtro de texto y ejecución con Task.Run para no congelar la ventana (Microsoft, 2024b).

Reporte	Parámetros	Origen de datos
Estudiantes por estado	Estado; texto	LINQ sobre Estudiantes
Sesiones por rango de fechas	Desde; hasta; texto (incluye materia)	JOIN de Asistencias_Sesiones, Estudiantes, Niveles, Materias y Profesores
Evaluaciones por nota mínima	Nota mínima (punto o coma); texto	JOIN de Evaluaciones_Finales, Estudiantes, Materias y Profesores

Tabla 7. Reportes dinámicos implementados
Fuente: elaboración propia.

En los JOIN, EF Core usa solo las llaves de los catálogos; los nombres se leen del catálogo ADO.NET, lo que tolera diferencias de nombres de columna. El diseño visual es una rejilla con el contador «Registros encontrados». **Casos límite:** sin coincidencias, la rejilla queda vacía y el contador muestra 0; exportar sin datos responde «Primero genere un reporte con datos.»; «Desde» no puede ser posterior a «Hasta»; y la nota mínima debe estar entre 0 y 100.

12. Manejo de errores y Logger

No se definieron excepciones personalizadas: la capa de negocio lanza InvalidOperationException o ArgumentException con un mensaje descriptivo y los formularios las capturan para mostrarlas al usuario. En el acceso a datos, cada escritura se ejecuta en una transacción que se revierte ante cualquier fallo y relanza la excepción, sin ocultarla (Martin, 2009):

```
using (SqlTransaction tx = cn.BeginTransaction())
{
   try { /* ejecutar el procedimiento con parámetros */ tx.Commit(); }
   catch { tx.Rollback(); throw; }
}
```

En los servicios web se capturan tipos específicos (HttpRequestException, TaskCanceledException, JsonException) y en el registro de usuarios se distinguen ArgumentException, InvalidOperationException y un catch general en el nivel de presentación. **El sistema no implementa el Logger** en archivo de texto que describe la guía; los errores se muestran con MessageBox y no se conservan. El catálogo de mensajes y sus causas está en el capítulo 21.

13. Pruebas realizadas

La Tabla 8 presenta los casos de prueba derivados de los requerimientos, con casos negativos y límite. Las columnas «Obtenido» y «Estado» deben completarse tras ejecutar cada caso en el sistema compilado; las capturas de evidencia de los casos CP-02, CP-08 y CP-13 se encuentran en el capítulo 21.

Cód.	Descripción	Pasos	Entrada	Resultado esperado
CP-01	Login válido (RF-01)	Ingresar credenciales → Ingresar	Usuario y clave correctos	Se abre el menú principal
CP-02	Login inválido (RF-01)	Ingresar clave errónea	Clave incorrecta	«Usuario o contraseña incorrectos.»
CP-03	Crear cuenta válida (RF-02)	Crear cuenta nueva → completar → Crear cuenta	Usuario nuevo; clave de 10+ con letras y números	Mensaje de cuenta creada; puede iniciar sesión
CP-04	Contraseña débil (RF-02)	Crear cuenta con clave corta	Clave de 6 caracteres	«La contraseña debe tener al menos 10 caracteres e incluir letras y números.»

Cód.	Descripción	Pasos	Entrada	Resultado esperado
CP-05	Usuario duplicado (RF-02)	Crear cuenta con un usuario existente	Usuario ya registrado	Se rechaza: el usuario ya existe
CP-06	Estudiante sin código (RF-03)	Dejar el código vacío → Agregar	Código vacío	«El código del estudiante es obligatorio.»
CP-07	Eliminar estudiante con registros (RF-03)	Seleccionar estudiante con sesiones → Eliminar	Estudiante con sesiones	La llave foránea lo impide; mensaje claro
CP-08	Sesión con fecha futura (RF-04)	Elegir fecha posterior a hoy → Agregar	Fecha futura	«La fecha de la asistencia no puede ser futura.»
CP-09	Evaluación fuera de rango (RF-05)	Pruebas = 150 → Agregar	Pruebas = 150	Se rechaza: valores de 0 a 100
CP-10	Nota automática (RF-05)	Ingresar los tres puntajes	10, 20 y 30	Nota final = 60
CP-11	Carga concurrente y cancelación (RF-06)	Iniciar proceso; abrir otro módulo; repetir y Cancelar	—	La interfaz responde; al cancelar se informa
CP-12	Consultar feriados (RF-10)	País NI, año 2026 → Consultar	NI / 2026	Tabla, próximo feriado y JSON
CP-13	Consulta sin Internet (RF-10)	Desconectar la red → Consultar	NI / 2026	«No se pudo conectar con el servicio web…»
CP-14	Correo sin clave API (RF-12)	Sin RESEND_API_KEY → Enviar notificación	Datos válidos	«No está configurada la variable de entorno RESEND_API_KEY…»
CP-15	Exportar sin datos (RF-09)	Exportar CSV sin generar reporte	—	«Primero genere un reporte con datos.»

Tabla 8. Casos de prueba
Fuente: elaboración propia.

14. Conclusiones técnicas

La Tabla 9 contrasta cada objetivo del capítulo 3 con el resultado, y explica la causa cuando el cumplimiento fue parcial.

Objetivo	Estado	Sustento técnico
1. Interfaz y registro de cuentas	Cumplido	Nueve formularios con validaciones, navegación y registro con PBKDF2.
2. CRUD con ADO.NET	Cumplido	Tres módulos con procedimientos almacenados, parámetros y transacciones.
3. Capas	Parcial	Separación en carpetas y formularios sin SQL; sin interfaces ni inyección de dependencias.
4. Concurrencia	Cumplido	Task, Thread, CancellationToken, IProgress e Interlocked.
5. EF Core, LINQ y reportes	Parcial	Reportes LINQ con CSV funcionan; sin PDF ni migraciones Code-First.
6. Servicios REST	Cumplido	Nager.Date y Resend con manejo de errores; sin reintentos.

Tabla 9. Cumplimiento de los objetivos específicos
Fuente: elaboración propia.

Lecciones aprendidas. La separación en capas permitió agregar EF Core, el registro de usuarios y las APIs sin modificar los módulos existentes. Mantener ADO.NET y EF Core sobre la misma base exigió convertidores de tipo y leer los nombres de catálogo por ADO.NET. Aceptar decimales con punto o coma fue necesario porque dependía de la configuración regional. Con una

base local las consultas son instantáneas, por lo que se simuló una carga pesada para observar la cancelación.

Recomendaciones. (1) Definir interfaces para los DAO e incorporar inyección de dependencias; (2) implementar un Logger en archivo; (3) agregar roles y una pantalla para cambiar contraseña, y migrar el usuario inicial a PBKDF2; (4) mover la cadena de conexión a appsettings.json y alinear los tipos del DbContext con el esquema real; (5) agregar pruebas unitarias de la capa de negocio y reintentos en los servicios web; (6) si se requiere, generar reportes en PDF.

Parte II. Manual de Usuario

15. Introducción y público objetivo

El Sistema de Gestión Académica es una aplicación de escritorio para registrar estudiantes, sesiones de asistencia y evaluaciones finales de un centro educativo, calcular la nota final automáticamente, generar reportes y consultar feriados. Está dirigido al personal administrativo o docente que lo usa a diario y a la persona que lo instala. Con él se puede registrar, modificar, eliminar y buscar información, exportar reportes a CSV y enviar notificaciones por correo; no permite administrar roles ni cambiar contraseñas, ni genera reportes en PDF.

La Tabla 10 resume los perfiles de uso.

Perfil	Funcionalidades disponibles
Usuario autenticado (todas las cuentas)	Estudiantes, Sesiones, Calificaciones, Programación concurrente, ORM y reportes, Servicios web y notificaciones por correo, Cerrar sesión.
Persona sin cuenta	Solo el inicio de sesión y la opción «Crear cuenta nueva».

Tabla 10. Perfiles de usuario
Fuente: elaboración propia.

16. Requisitos del sistema

La Tabla 11 presenta los requisitos para ejecutar la aplicación.

Requisito	Mínimo / recomendado
Sistema operativo	Windows 10 o superior.
Prerrequisito de ejecución	.NET 8 Desktop Runtime (https://dotnet.microsoft.com/download/dotnet/8.0).
Base de datos	SQL Server 2019 Express o superior, instancia SQLEXPRESS (https://www.microsoft.com/sql-server/sql-server-downloads), y SQL Server Management Studio para ejecutar el script.
Memoria RAM	Mínimo 4 GB; recomendado 8 GB.
Espacio en disco	El necesario para SQL Server Express y la aplicación; la base de datos del proyecto es pequeña.
Internet	Solo para consultar feriados y enviar correos; las consultas son livianas y no requieren gran ancho de banda.
Correo (opcional)	Cuenta de Resend y variable de entorno RESEND_API_KEY (opcionalmente RESEND_FROM_EMAIL).

Tabla 11. Requisitos del sistema
Fuente: elaboración propia.

17. Procedimiento de instalación

1. **Instalar los prerrequisitos:** el .NET 8 Desktop Runtime y SQL Server Express con SQL Server Management Studio (SSMS).

2. **Crear la base de datos:** abrir SSMS, conectarse a localhost\SQLEXPRESS, abrir el archivo Database/Script_Instalacion_Completo.sql y pulsar Ejecutar. El script crea las tablas, los catálogos mínimos, el usuario inicial y los procedimientos, y no borra datos existentes. Si ya tenía una versión anterior de la base, ejecute además Database/Actualizar_Seguridad_Usuarios.sql una sola vez.
3. **Revisar la conexión:** la aplicación busca la instancia localhost\SQLEXPRESS y la base «Gestion Academica». Si su instancia es distinta, hay que editar DataAccess/Conexion.cs y volver a compilar (no existe appsettings.json).
4. **Copiar el ejecutable:** copiar la carpeta de salida compilada en Release (GestionAcademica.exe y sus dependencias) al equipo del usuario.
5. **Configurar el correo (opcional):** crear la variable de entorno de Windows RESEND_API_KEY con su clave de Resend (y, si desea, RESEND_FROM_EMAIL) y reiniciar la aplicación.
6. **Ejecutar GestionAcademica.exe:** debe aparecer la pantalla de inicio de sesión (véase la Figura 7).

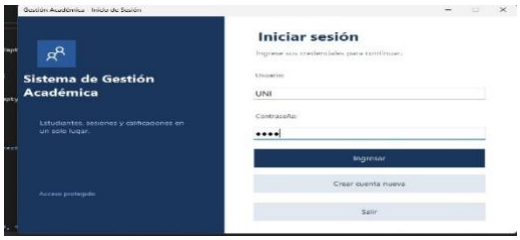
18. Credenciales de acceso y roles

El script crea el usuario inicial **UNI**; su contraseña es la que usa su equipo:

El sistema no incluye una pantalla para cambiar contraseñas. Por seguridad se recomienda crear una cuenta propia (capítulo 20, caso 1) y, si se desea, desactivar la cuenta UNI en la base de datos (columna Activo = 0). No existen roles: todas las cuentas acceden a las mismas funciones.

19. Guía de navegación por pantallas

Inicio de sesión. Contiene los campos Usuario y Contraseña y los botones Ingresar, Salir y «Crear cuenta nueva» (Figura 7). **Crear cuenta.** Pide nombre completo, nombre de usuario (4 a 50 caracteres: letras, números, punto, guion o guion bajo), contraseña de al menos 10 caracteres con letras y números, y su confirmación (Figura 8).



Menú principal. Muestra el reloj, el botón «Cerrar sesión» y dos grupos: «Gestión académica» (Estudiantes, Sesiones, Calificaciones) y «Procesos y reportes» (Programación concurrente, ORM y reportes, Servicios web)



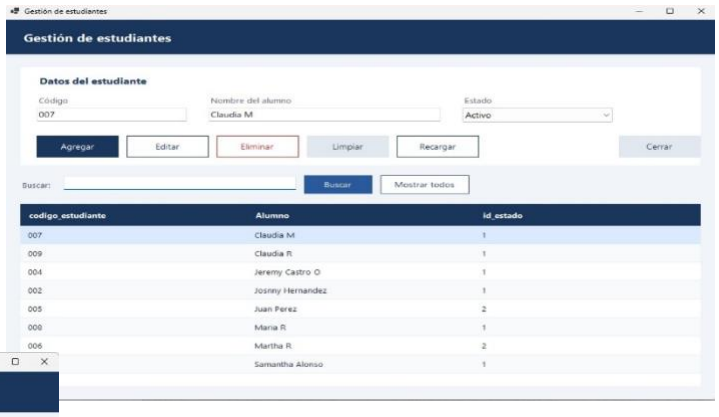
Figura 9. Menú principal

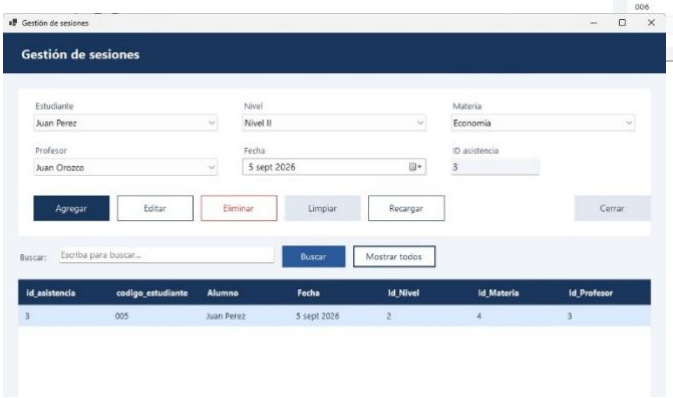
Fuente: captura de pantalla del sistema ejecutable (pendiente de insertar).

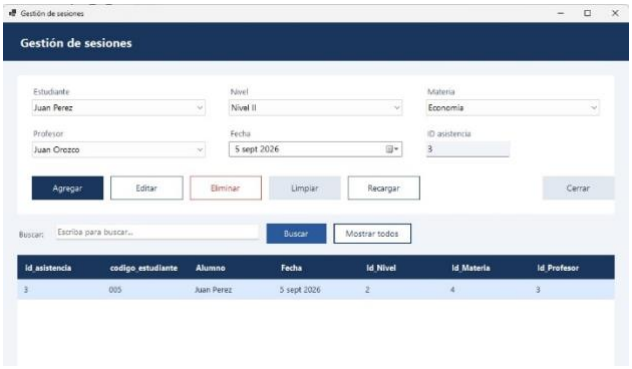
Formularios de mantenimiento. Estudiantes, Sesiones y Calificaciones comparten la misma disposición: campos de captura arriba y rejilla de resultados abajo. Los botones son

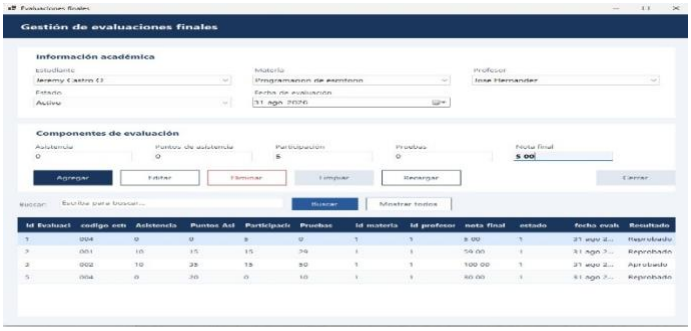


Agregar (guarda un registro nuevo), **Modificar** (guarda cambios del registro seleccionado), **Eliminar** (pide confirmación), **Limpiar** (vacía los campos), **Actualizar** (recarga la rejilla), **Buscar** (filtra por el texto escrito; también con Enter), **Mostrar todos** y **Cerrar**. En Calificaciones, la nota final se calcula sola

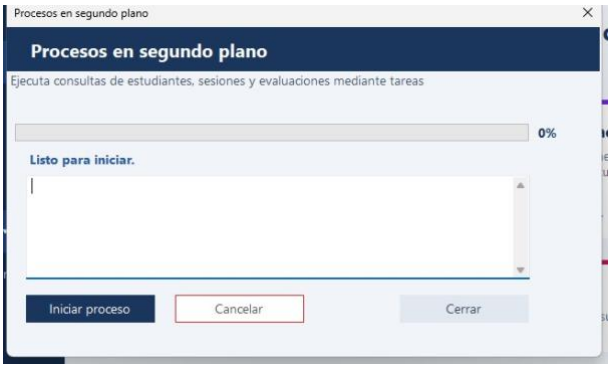

Gestión de estudiantes
Gestión de estudiantes
Datos del estudiante
Código
007
Nombre del alumno
Claudia M
Estado
Activo
Agregar
Editar
Eliminar
Limpiar
Recargar
Cerrar
Buscar:
Buscar
Mostrar todos
codigo_estudiante
Alumno
id_estado
007
Claudia M
1
009
Claudia R
1
004
Jeremy Castro O
1
002
Josnny Hernandez
1
005
Juan Perez
2
000
Maria R
1
006
Martha R
2
Samantha Alonso
1


Gestión de sesiones
Gestión de sesiones
Estudiante
Juan Perez
Nivel
Nivel II
Materia
Economia
Profesor
Juan Orozco
Fecha
5 sept 2026
ID asistencia
3
Agregar
Editar
Eliminar
Limpiar
Recargar
Cerrar
Buscar:
Escriba para buscar...
Buscar
Mostrar todos
id_asistencia
codigo_estudiante
Alumno
Fecha
Id_Nivel
Id_Materia
Id_Profesor
3
005
Juan Perez
5 sept 2026
2
4
3


Gestión de sesiones
Gestión de sesiones
Estudiante
Juan Perez
Nivel
Nivel II
Materia
Economia
Profesor
Juan Orozco
Fecha
5 sept 2026
ID asistencia
3
Agregar
Editar
Eliminar
Limpiar
Recargar
Cerrar
Buscar:
Escriba para buscar...
Buscar
Mostrar todos
Id_asistencia
codigo_estudiante
Alumno
Fecha
Id_Nivel
Id_Materia
Id_Profesor
3
005
Juan Perez
5 sept 2026
2
4
3


Gestión de evaluaciones finales
Información académica
Componentes de evaluación
Agregar
Editar
Eliminar
Limpiar
Recargar
Cerrar
Buscar
Mostrar todos

Programación concurrente. «Iniciar proceso» carga estudiantes, sesiones y evaluaciones en segundo plano con barra de progreso; «Cancelar» lo detiene. Mientras corre, puede seguir usando otros módulos (Figuras 13 y 14). **ORM y reportes** y **Servicios web** se explican en los casos de uso del capítulo 20.



20. Casos de uso paso a paso

Caso 1. Crear una cuenta nueva

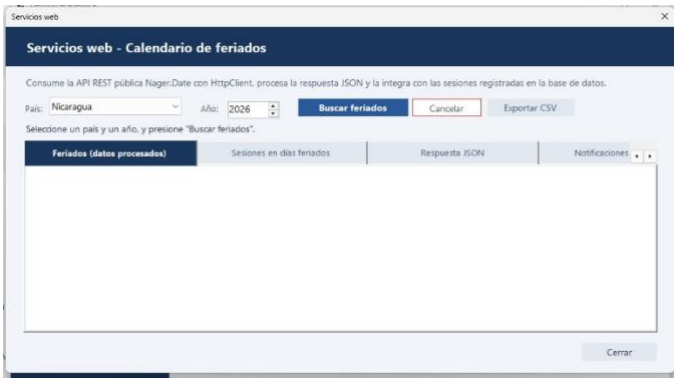
Elemento	Descripción
Objetivo / actor	Registrar un usuario para iniciar sesión / persona sin cuenta.
Precondiciones	Aplicación abierta en el inicio de sesión; base de datos instalada y actualizada.
Pasos	1) Pulsar «Crear cuenta nueva». 2) Escribir nombre completo y nombre de usuario. 3) Escribir la contraseña (10 o más caracteres con letras y números) y confirmarla. 4) Pulsar «Crear cuenta».
Postcondiciones	Aparece «La cuenta se creó correctamente» y puede iniciar sesión con el nuevo usuario.
Posibles errores	«Las contraseñas no coinciden.»; «Ese nombre de usuario ya existe. Elegí otro.»; datos fuera de las reglas de longitud o formato.

Caso 2. Registrar una evaluación final (transacción principal)

Elemento	Descripción
Objetivo / actor	Guardar el resultado final de un estudiante / usuario autenticado.
Precondiciones	Existen el estudiante, la materia, el profesor y el estado en el sistema.
Pasos	1) Menú → Calificaciones. 2) Elegir estudiante, materia, profesor y estado. 3) Escribir asistencia, puntos de asistencia, participación y pruebas (0 a 100). 4) Verificar la nota final calculada. 5) Pulsar Agregar.
Postcondiciones	La evaluación aparece en la rejilla y queda guardada en la base de datos.
Posibles errores	Algún valor fuera de 0 a 100 o campo sin elegir: se muestra el mensaje de validación y no se guarda.

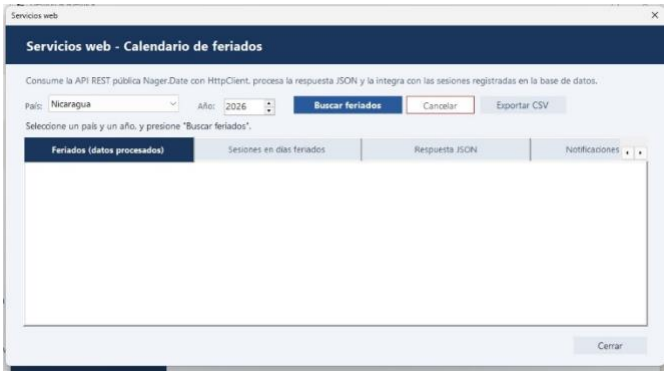
Caso 3. Consultar feriados (API externa)

Elemento	Descripción
Objetivo / actor	Ver los feriados de un país y detectar sesiones en días no laborables / usuario autenticado.
Precondiciones	Conexión a Internet.
Pasos	1) Menú → Servicios web. 2) Elegir país y año. 3) Pulsar Consultar. 4) Revisar las pestañas «Feriados», «Sesiones en días feriados» y «Respuesta JSON». 5) Opcional: Exportar a CSV.
Postcondiciones	Se muestran los feriados, el próximo feriado y las sesiones coincidentes.
Posibles errores	Sin Internet o tiempo agotado; año fuera de 2000–2100; país sin datos.



Caso 4. Generar un reporte y exportarlo a CSV

Elemento	Descripción
Objetivo / actor	Obtener un listado filtrado y guardarlo en un archivo / usuario autenticado.
Precondiciones	Existen datos registrados.
Pasos	1) Menú → ORM y reportes → pestaña Reportes. 2) Elegir el tipo (estudiantes por estado, sesiones por fechas o evaluaciones por nota mínima) y los filtros. 3) Pulsar Generar reporte y revisar «Registros encontrados». 4) Pulsar Exportar CSV y elegir dónde guardar.
Postcondiciones	Se crea un archivo CSV que puede abrirse con Excel.
Posibles errores	«Desde» posterior a «Hasta»; nota mínima fuera de 0 a 100; exportar sin generar primero el reporte.



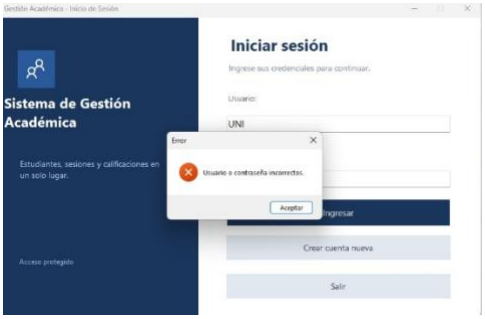
Caso 5. Enviar una notificación por correo

Elemento	Descripción
Objetivo / actor	Avisar por correo a un estudiante o tutor / usuario autenticado.
Precondiciones	Variable RESEND_API_KEY configurada e Internet disponible.
Pasos	1) Menú → Servicios web → pestaña «Notificaciones por correo». 2) Escribir correo del destinatario, nombre del estudiante, asunto y mensaje. 3) Pulsar «Enviar notificación».
Postcondiciones	Se muestra el identificador devuelto por el servicio. Esto indica que fue aceptado, no que llegó a la bandeja.
Posibles errores	Falta la clave API; correo inválido; campos vacíos o demasiado largos.

21. Errores comunes y soporte

La Tabla 12 reúne los mensajes más frecuentes. El sistema no genera un archivo de registro (log); ante un error persistente, anote el mensaje exacto, tome una captura de pantalla y envíela al soporte técnico.

Mensaje	Causa probable	Acción
Debe ingresar usuario y contraseña.	Campos vacíos.	Completar ambos campos.
Usuario o contraseña incorrectos.	Datos erróneos o cuenta inactiva.	Revisar los datos o crear una cuenta.
No se pudo validar el usuario contra la base de datos…	SQL Server detenido o instancia distinta.	Iniciar SQL Server y revisar Conexion.cs.
Ese nombre de usuario ya existe. Elegí otro.	Usuario repetido.	Elegir otro nombre de usuario.
La contraseña debe tener al menos 10 caracteres…	Contraseña débil.	Usar 10 o más caracteres con letras y números.
No se pudo registrar la cuenta… script de actualización	Base sin actualizar.	Ejecutar Actualizar_Seguridad_Usuarios.sql.
El código del estudiante es obligatorio.	Código vacío.	Escribir el código.
La fecha de la asistencia no puede ser futura.	Fecha posterior a hoy.	Elegir una fecha válida.
No se pudo conectar con el servicio web…	Sin Internet.	Revisar la conexión y reintentar.
No está configurada la variable RESEND_API_KEY…	Falta la clave de correo.	Crear la variable de entorno y reiniciar.
Primero genere un reporte con datos.	Exportar sin reporte.	Generar el reporte antes de exportar.



22. Glosario de términos

Término	Definición
API REST	Servicio en Internet al que la aplicación hace solicitudes (capítulo 20, casos 3 y 5).
CRUD	Crear, consultar, modificar y eliminar registros.
CSV	Archivo de texto con datos separados por punto y coma, abrible con Excel.
Evaluación final	Registro con asistencia, participación, pruebas y nota final de un estudiante por materia.
Feriado	Día no laborable oficial de un país.
Nota final	Suma de puntos de asistencia, participación y pruebas, calculada por el sistema.
PBKDF2	Técnica que protege las contraseñas nuevas para que no se guarden en texto plano.
Sesión	Registro de asistencia de un estudiante en una fecha, nivel, materia y profesor.
SQL Server Express	Programa gratuito donde se guarda la base de datos del sistema.

.

23. Preguntas frecuentes

¿Qué hago si no recuerdo mi contraseña? El sistema no tiene recuperación; cree una cuenta nueva (caso 1).

¿Puedo cambiar mi contraseña? No hay una pantalla para ello; cree una cuenta nueva con otra contraseña.

¿Cómo instalo la base de datos? Ejecute Script_Instalacion_Completo.sql en SSMS (capítulo 17).

¿Por qué dice que no puede conectar con la base de datos? SQL Server no está iniciado o su instancia no es SQLEXPRESS (capítulo 21).

¿Se pierden los datos al ejecutar de nuevo el script? No; el script solo crea lo que falta.

¿Cómo se calcula la nota final? Suma de puntos de asistencia, participación y pruebas.

¿Por qué no puedo guardar una sesión? La fecha no puede ser futura y todos los campos son obligatorios.

¿Puedo cerrar la ventana durante un proceso concurrente? Sí, pero se cancelará el proceso y se le avisará.

¿Los reportes se guardan en PDF? No; se exportan a CSV (caso 4).

¿Por qué no llega el correo? Un 200 solo indica que Resend aceptó la solicitud; revise spam, el remitente y que la clave sea válida.

¿Funciona sin Internet? Sí, salvo feriados y correo.

Bibliografía

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Martin, R. C. (2009). *Clean code: A handbook of agile software craftsmanship*. Prentice Hall.

Martin, R. C. (2018). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Microsoft. (2024a). *Documentación oficial de .NET*. https://learn.microsoft.com/dotnet

Microsoft. (2024b). *Entity Framework Core — Documentación oficial*. https://learn.microsoft.com/ef/core

Microsoft. (2024c). *Patrones de diseño asincrónico en C#.* https://learn.microsoft.com/dotnet/csharp/asynchronous-programming

Microsoft. (s. f.-a). *Documentación de SQL Server*. https://learn.microsoft.com/sql/

Microsoft. (s. f.-b). *Windows Forms: documentación de .NET Desktop*. https://learn.microsoft.com/dotnet/desktop/winforms/

Nager.Date. (s. f.). *Public holiday API*. https://date.nager.at/Api

Resend. (s. f.). *Send email: API reference*. https://resend.com/docs/api-reference/emails/send-email